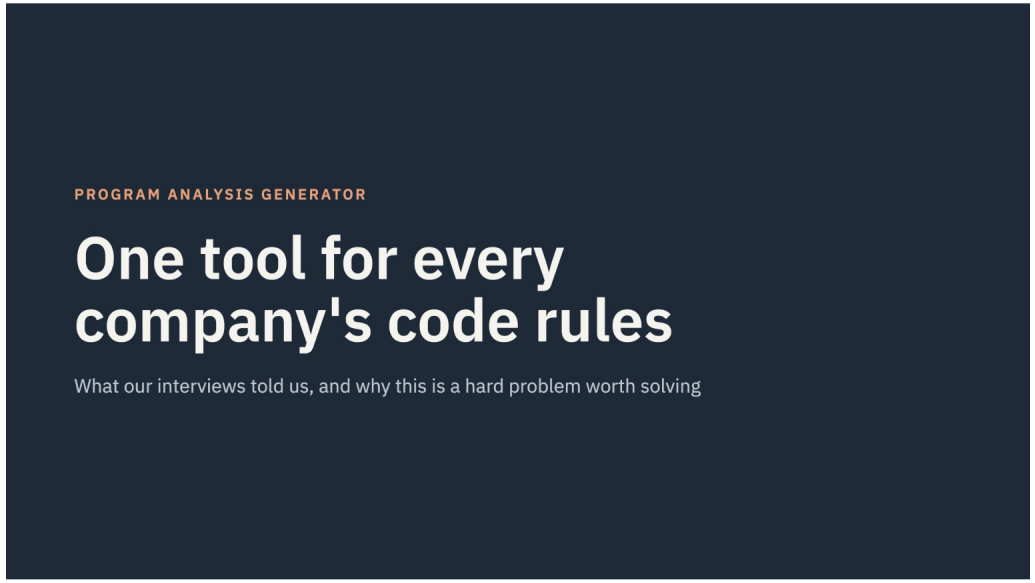


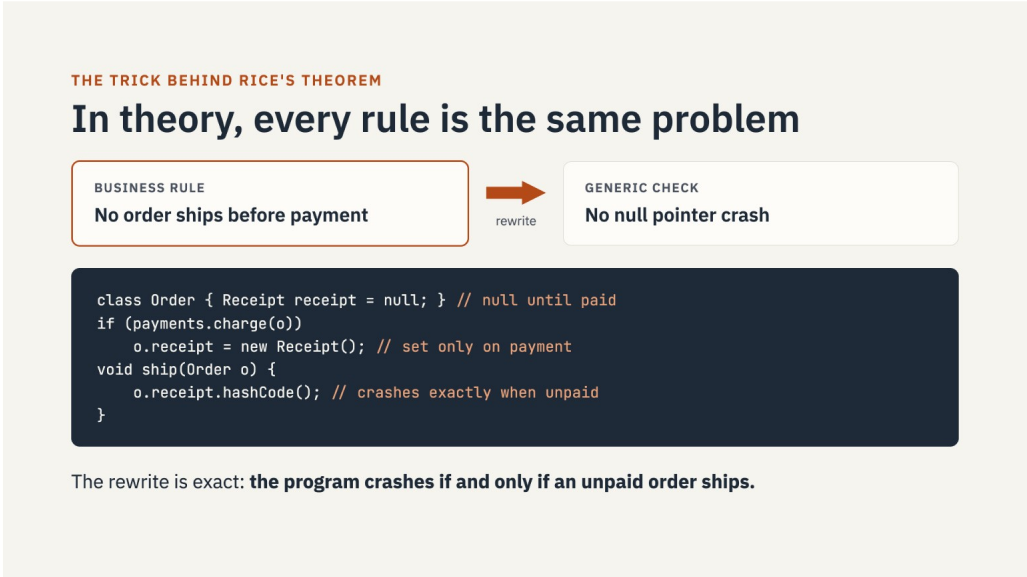
Outline: what the market wants, why it has been hard to serve, why in theory it is one problem, the catch, and our idea.



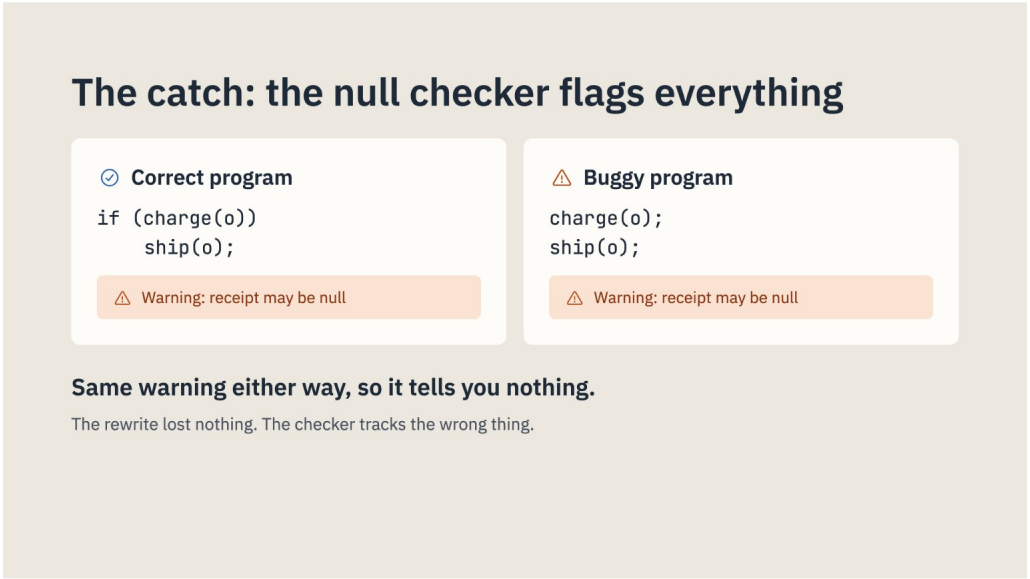
Many developers want to spend less time reviewing code, especially AI-generated code, and want tools that check specific rules automatically. These six came up in our interviews. Temporal: replaying a workflow must produce the same result, or production breaks. Data races: e.g. a resource used before it is created. The order-and-payment rule is the running example for the rest of the talk.



Classically, each rule needs its own purpose-built checker, written by program analysis experts. Each rule is specific to one company or one framework. Deep Dive: a 10-person team at Google built the infrastructure for hard-coded keys and a few similar Android properties. Temporal has its own team building a lint tool for the call-graph-related rules. So the market is a long tail of niches, and each niche costs an expert team to serve.



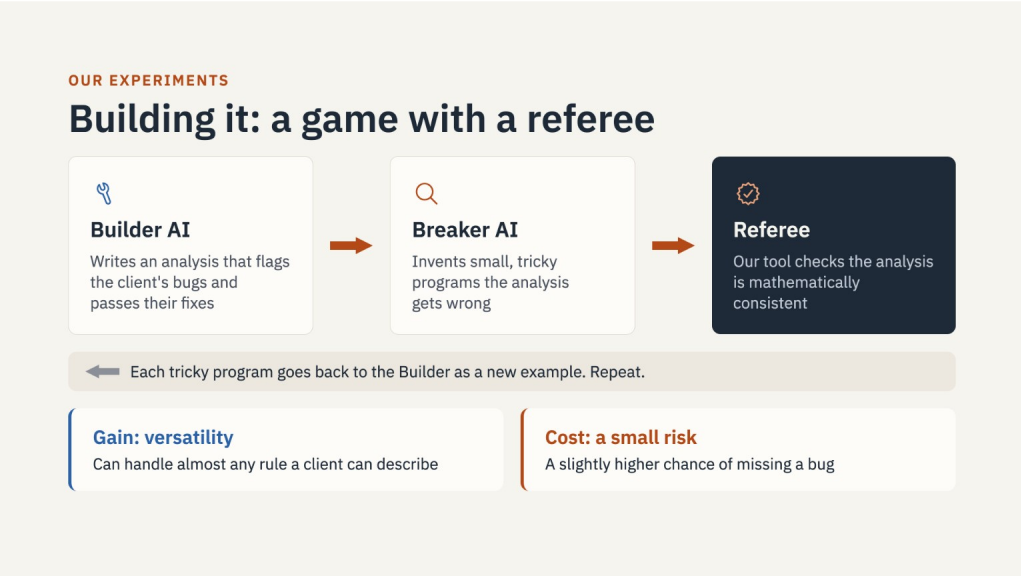
Rice's theorem says every interesting question about what a program does is undecidable: no tool answers it perfectly for all programs. The proof works by translating one question about program behavior into another. We can use the same trick in practice: rewrite a program so a company-specific rule becomes a generic one that existing tools already check. Here, the receipt field is added only for the check. So in principle, any off-the-shelf null pointer checker, a mature and widely deployed kind of tool, can now check a business rule.



In practice, every modern null pointer checker flags ship on every program, correct or not. It sees a field that starts out null and is only sometimes set, and has no way to connect "non-null here" to "charge returned true for this order." The checker's abstraction, its built-in idea of what to keep track of, is wrong for this rule. A purpose-built checker would track each order's payment status, and nothing else. That is why each rule has needed its own expert team: picking the abstraction is the expert work.



A quality or security engineer writes the rewrite, which needs no analysis expertise, and our tool finds the right abstraction for it automatically. One product serves the whole long tail of company-specific rules.



The AI does the hard, creative work, and it gets checked carefully. The client gives us examples of their bugs and the fixes. The Builder writes an analysis (technically, an abstract domain) that flags the bugs and passes the fixes. The Breaker tries to fool it with small, adversarial programs; each one becomes a new example for the Builder. The Referee is our tool, not an AI: it checks that the analysis is mathematically consistent. The trade-off: this lets us handle the arbitrary rewrites from the earlier slides, at the cost of a slightly higher chance of being unsound, meaning it could miss a bug. Our experiments: we run this on local open models of several sizes to learn how small a model can still play the game well.